

3.1 Introduction

This chapter defines the engineering methodology used to design, implement, and evaluate a hybrid detection system for AI-generated steganographic malware in images. The methodology is expressed as a repeatable pipeline: (1) construct and label datasets, (2) run a two-stage detector (YARA → ML anomaly detection), (3) log per-file decisions and intermediate outputs, and (4) compute performance metrics and error analyses from the logs.

The proposed system is implemented as a Python-driven batch processing framework that orchestrates image ingestion, YARA scanning, feature extraction, anomaly scoring, and decision fusion. The system is executed in an isolated environment (VirtualBox hosted Ubuntu/Kali) to ensure safe handling and reproducibility

Siphuma TS_217001561_ - Copy

. External benchmarking is performed using the VirusTotal API to obtain multi-engine scan summaries (as a reference point), not to rank products

Siphuma TS_217001561_ - Copy

.

The remainder of this chapter specifies *exactly* how each system component operates: threat assumptions (what attacks the system targets), dataset construction and labelling (how ground truth is defined), feature engineering (what signals the ML model consumes), detection logic (what each stage outputs), evaluation (how TP/FP/TN/FN are produced), and the step-by-step procedure for inspecting errors (why misclassifications happened).

3.2 Research Paradigm and Approach

This study uses an **experimental engineering approach** because the objective is to build a detection system and measure its effectiveness under controlled conditions. The “data” is not opinions; it is system output logs generated when the pipeline processes labelled files. The approach is therefore: run the detector on known inputs, capture outputs, and calculate performance measures from those outputs.

How the system produces measurable outputs (per file):

1. The pipeline reads an image file and assigns its **ground-truth label** (true_label) based on dataset membership (e.g., benign folder vs stego-malware folder).
2. The YARA module runs and outputs $yara_hit \in \{0,1\}$ plus optional rule names (if matched).

3. The feature extractor computes a numeric vector x for that file.
4. The ML module outputs a continuous score ml_score and a binary decision ml_pred (based on thresholding).
5. The fusion module outputs the final decision $final_pred \in \{0,1\}$ plus a $decision_reason$ (e.g., “YARA match”, “ML threshold”, “benign”).

What gets logged (this is what makes the study “proved” and reproducible):

- A row-per-file results table (CSV/JSON) with at least:
 $file_id$, $true_label$, $yara_hit$, $matched_rules$, ml_score , ml_pred , $final_pred$, $decision_reason$, $runtime_ms$
- A run manifest: tool versions, rule-set hash, model version, threshold used, dataset split IDs.

This design ensures that every claim you make later (accuracy, FP rate, etc.) can be traced back to raw logged decisions rather than narrative description.

3.3 Threat Model and Assumptions

The system targets adversaries who hide malicious content inside seemingly benign image files using steganography, including AI-assisted variants. In scope, the attacker’s goal is **evasion of static antivirus** by making the carrier file look normal to conventional scanners. The defender (your system) operates pre-execution: it inspects a file before it is opened or executed.

System boundary (what your detector sees):

- The detector sees the **image file bytes**, decoded pixel data, file structure, and metadata (where relevant).
- The detector does **not** rely on live network traffic, sandbox execution traces, or C2 callbacks (those are out of scope for this thesis).

Operational assumptions used to make the experiments valid:

1. You can build labelled datasets where the ground truth is known (benign vs stego-malware).
2. The detector runs in an isolated lab environment (VirtualBox on Ubuntu/Kali)

Siphuma TS_217001561_ - Copy

.

3. VirusTotal API results can be used as a comparative reference scan summary (multi-engine signal)

Siphuma TS_217001561_ - Copy

.

Scope limitations (stated explicitly so examiners don't attack you for it):

- Focus is **image-based** steganography (not audio/video).
- Detection is primarily **static** (no dynamic execution).
- The study evaluates a hybrid detector that combines signature rules + statistical anomaly detection, not a full end-to-end antivirus product.

3.4 Data Sources and Dataset Constructions

This section defines **exactly** how you create the datasets that drive training and evaluation and how the system obtains **ground truth** (so TP/FP/TN/FN are mathematically valid).

Dataset categories

Your system requires at least:

1. **Benign images (B):** clean images with no hidden payloads.
2. **Steganographic test images (S):** images containing embedded content representative of stego-malware behaviour.

Ground-truth labelling (where truth comes from)

Ground truth is assigned by construction:

- All files in benign set B $\rightarrow$ true_label = 0
- All files in stego set S $\rightarrow$ true_label = 1

This removes ambiguity: the “truth” is not guessed by VirusTotal or YARA; it is defined by how you built the dataset.

Construction procedure (system steps)

Step 1 — Acquire benign images (B):

- Collect a diverse set of benign images across sizes, formats, and compression levels.
- Record format distribution to avoid a biased dataset.

Step 2 — Acquire malicious references (safe research handling):

- You may use public security repositories or known samples in a controlled lab, but the methodology should emphasize **safe handling** and not include operational details that enable misuse.

Step 3 — Create steganographic images (S):

- Use steganographic tooling (e.g., Steghide) and custom Python scripts to embed a controlled payload into benign carriers

Siphuma TS_217001561_ - Copy

.

- **Safe thesis practice:** use *non-operational payloads* (e.g., inert byte patterns or encrypted random blobs) that mimic embedding artefacts without creating runnable malware. This keeps the research defensive while still scientifically valid.

Step 4 — Preprocessing and standardization:

- Normalize formats where appropriate (e.g., keep consistent color spaces, consistent resizing strategy only if justified).
- Remove corrupted files and log exclusions.

Step 5 — Dataset split and indexing:

- Split into train/validation/test using a fixed seed.
- Store split manifests (file lists) so the same experiment can be rerun.

Step 6 — Build a dataset index:

- The system creates an index file mapping file_id → path → true_label → split.

3.5 Feature Engineering and Analysis Strategy

This section explains **what features the system extracts, how they are extracted, and why** those features support both anomaly detection and analysis.

Feature extraction pipeline (system steps)

For each image file:

1. **Load and decode** image into a numeric pixel array (e.g., via OpenCV/Pillow).
2. **Compute pixel-statistics features**, such as per-channel mean/variance, higher-order moments, histogram summaries.

3. **Compute entropy and information-density features** (global entropy; optionally local/patch entropy).
4. **Compute structural/file-level features** from raw bytes where justified: file size, header consistency, marker distribution, metadata presence.
5. **Combine features** into a single vector $x \in \mathbb{R}^d$.
6. **Normalize** features (e.g., z-score scaling) using training-set statistics only.
7. **Persist** features to disk (e.g., features.parquet/csv) linked by file_id.

Analysis strategy (how you show it works)

- Before training ML, compute summary statistics per feature for benign vs stego sets to confirm separability signals exist.
- Use correlation/variance filtering to remove redundant features.
- The output of this section is a **documented feature set** and a reproducible feature extraction script, so later results can be traced back.

3.6 Signature-Based Detection Methodology

This section explains how the **YARA stage** is implemented, what it outputs, and how it drives the pipeline. Your tool choice is explicitly supported by your document: YARA for rule development/testing

Siphuma TS_217001561_ - Copy

.

YARA integration (system steps)

1. **Rule authoring:** Create a rule set targeting indicators relevant to stego-malware artefacts (e.g., suspicious byte sequences, abnormal markers, known tool signatures).
2. **Rule compilation:** Compile rules into a versioned ruleset and store a hash of the file (for reproducibility).
3. **Scan execution:** For each input file, run YARA scanning and record:
 - yara_hit = 1 if any rule matches else 0
 - matched_rules = [rule_name1, rule_name2, ...]
4. **Stage output:** Write YARA results into the per-file results table.
5. **Routing logic:**

- If `yara_hit == 1`, the file is flagged and either (a) stops (fast path) or (b) still goes to ML for confirmation depending on your chosen fusion design (you define this explicitly in 3.8).

Proving this stage works

- Run YARA on labelled datasets and compute standalone confusion matrix for the YARA stage alone (baseline).
 - Record which rules contribute most to FPs and FNs.
-

3.7 Machine Learning–Based Anomaly Detection Methodology

This section explains how the anomaly model is trained, how scores are computed, and how thresholds become decisions. Your document states ML frameworks like TensorFlow/PyTorch

Siphuma TS_217001561_ - Copy

.

Model design goal

- Learn “normal” characteristics from benign images.
- Flag deviations likely caused by embedding artefacts.

Training procedure (system steps)

1. **Prepare training features:** Use only benign training split features.
2. **Select model family:** Choose anomaly model(s) (e.g., Isolation Forest or autoencoder).
3. **Fit the model:** Train on benign feature vectors.
4. **Calibrate threshold:**
 - Score a benign validation set, choose threshold T such that FP rate on benign validation is within an acceptable bound (e.g., 1–5%).
 - Save T alongside the model.

Detection procedure (system steps)

For each file:

1. Extract features x .
2. Compute anomaly score $s = f(x)$.

3. Convert to binary decision:
 - $ml_pred = 1$ if $s > T$ else 0
4. Log: $ml_score=s$, $threshold=T$, ml_pred .

Proving ML stage works

- Compute metrics for ML stage alone.
 - Plot score distributions for benign vs stego sets (in Chapter 5) to justify the threshold.
-

3.8 Hybrid Detection Pipeline Design

This is where you show the **full system workflow** end-to-end, including decision fusion. Your document supports the hybrid approach as the core design

Siphuma TS_217001561_ - Copy

.

End-to-end pipeline (system steps)

Input: $file_id$, $file_path$, $true_label$ from dataset index.

1. **Ingest file** → compute a file fingerprint (hash) for tracking.
2. **Stage 1 (YARA):**
 - Run YARA scan → output $yara_hit$, $matched_rules$.
3. **Feature extraction:**
 - Extract numeric feature vector x .
4. **Stage 2 (ML anomaly):**
 - Compute ml_score → produce ml_pred .
5. **Fusion decision:**
 - Define explicit fusion rule (example, simple and defensible):
 - If $yara_hit == 1$ → $final_pred = 1$ (known-indicator strong signal)
 - Else if $ml_score > T$ → $final_pred = 1$ (unknown/anomalous)
 - Else → $final_pred = 0$
 - Log $decision_reason$ as "YARA" / "ML" / "Benign".
6. **Write outputs:**

- Append row to results.csv.
- Copy misclassified samples to errors/FP and errors/FN after metrics are computed (see 3.9).

What this design achieves (you must state this explicitly)

- Speed: YARA can catch known patterns quickly.
- Coverage: ML catches unknown patterns.
- Reduced false positives: fusion rule + calibrated threshold reduces benign images being flagged.

3.9 Evaluation Metrics and Performance Criteria

This section is where you answer your exact question: **what metrics, how computed, from where**, plus runtime/scalability and VirusTotal comparative scanning

Siphuma TS_217001561_ - Copy

.

Confusion matrix logging (exactly how TP/FP/TN/FN are produced)

For each file you already have:

- $\text{true_label} \in \{0,1\}$
- $\text{final_pred} \in \{0,1\}$

You compute counts by iterating over all result rows:

- $\text{TP} = \text{count}(\text{true_label}=1 \text{ AND } \text{final_pred}=1)$
- $\text{FP} = \text{count}(\text{true_label}=0 \text{ AND } \text{final_pred}=1)$
- $\text{TN} = \text{count}(\text{true_label}=0 \text{ AND } \text{final_pred}=0)$
- $\text{FN} = \text{count}(\text{true_label}=1 \text{ AND } \text{final_pred}=0)$

Metric formulas (what the system calculates)

From TP/FP/TN/FN:

- **Accuracy** = $(\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{TN} + \text{FN})$
- **Precision** = $\text{TP} / (\text{TP} + \text{FP})$
- **Recall (TPR)** = $\text{TP} / (\text{TP} + \text{FN})$
- **F1-score** = $2\text{TP} / (2\text{TP} + \text{FP} + \text{FN})$

- **False Positive Rate** = $FP / (FP + TN)$
- **False Negative Rate** = $FN / (FN + TP)$

Performance criteria beyond accuracy (supported by your doc)

You explicitly mention false positive/negative assessment, processing overhead, and resilience against evasion

Siphuma TS_217001561_ - Copy

. Here is how the system measures them:

A) False positives / negatives

- Computed from FP and FN rates above.
- The system also outputs fp_list.csv and fn_list.csv listing file IDs.

B) Processing overhead

- For each file, the system logs:
 - yara_time_ms, feature_time_ms, ml_time_ms, total_time_ms
- Summaries reported as:
 - mean/median processing time per file
 - throughput (files/second) for batch scans

C) Resilience against evasion (method, not weaponization)

- Evaluate detector stability across variations of carrier images (compression levels, resolutions, formats) and embedding intensity levels (low/medium/high).
- The key is: does performance degrade sharply when images change?

VirusTotal comparative scanning (how it is used)

Using the VirusTotal API

Siphuma TS_217001561_ - Copy

, for each file the system records:

- vt_detection_count = number of engines flagging the sample
- vt_total_engines = total engines returned
- vt_ratio = count/total

This is used as a reference signal (“how many engines flagged it”), not as a ranking system.

3.10 Ethical and Safety Considerations

This section operationalizes safety: **where experiments run, how samples are handled, and how you avoid enabling misuse.** Your document states isolated VirtualBox Ubuntu/Kali

Siphuma TS_217001561_ - Copy

and the toolset

Siphuma TS_217001561_ - Copy

.

Safety controls (system steps)

1. Run all analysis inside isolated VirtualBox VMs with restricted permissions

Siphuma TS_217001561_ - Copy

.

2. Store samples in encrypted/controlled directories.
3. Do not execute unknown binaries; treat all suspicious content as hazardous.
4. Use inert payloads for generated samples where possible to keep research defensive.

Responsible use

- VirusTotal is used only for scan summaries and benchmarking, not to publish “best/worst AV” claims

Siphuma TS_217001561_ - Copy

.

- Methods are documented to support defense and reproducibility, while avoiding actionable “how-to” steps that enable harm.

3.11 Chapter Summary

This chapter defined the complete methodology used to build and evaluate the proposed hybrid steganographic malware detection system. It specified how datasets are constructed and labelled (ground truth), how YARA and ML modules generate predictions, how fusion produces final decisions, and how performance metrics are calculated directly from logged system outputs.

The chapter also established the evaluation criteria (false positives/negatives, overhead, resilience)

Siphuma TS_217001561_ - Copy

, the toolchain (YARA, VirusTotal API, ML frameworks, steganographic tools, Python scripts)

Siphuma TS_217001561_ - Copy

, and the isolated experimental environment

Siphuma TS_217001561_ - Copy

. Chapter 4 then details the concrete implementation of these components (environment setup, sample generation, rule/model implementation, and integration) as already structured in your document